

1 Introduction

Job scheduling is an important part of a distribution system, particularly in data centres where resources must be managed efficiently to handle dynamic workloads. In any environment where resource allocation is important, decisions about where and when to run jobs can significantly impact overall system performance. Scheduling ensures that jobs are completed in a timely manner but also that resources like cores, memory and network are utilised optimally. As data centre operations scale, the need for intelligent scheduling policies become more in demand to meet service level agreements and cost constraints.

The main purpose of this assignment is to design and implement a job scheduling algorithm that achieves an optimised average turnaround time while not sacrificing other performance metrics such as resource utilisation and server rental cost. These metrics reflect the responsiveness of the system and the degree in which available hardware resources are being used. By comparing the custom scheduler against several baseline algorithms including First Fit (FF), Best Fit (BF), Fastest Completion (FC) and Fastest Available Fastest Completion (FAFC), it can be demonstrated where measurable improvements in turnaround efficiency is obtained without significant trade-offs in utilisation or cost.[1, 2]

The algorithm designed in this project uses a strategy that evaluates each job's Estimated Job Wait Time (EJWT), a function provided by `ds-sim`, in conjunction with a penalty for over-provisioning server resources. This allows smarter placement decisions of jobs that reduce queuing delays while still considering long-term efficiency. The algorithm prioritises servers where the job is more likely to start sooner and finish faster. This helps avoid those jobs that might be idle but inefficient for the job type. By incorporating both queue-time predictions and resource penalties, the scheduler is adaptive and versatile across various system configurations.[1]

This report presents a comprehensive evaluation of the proposed client scheduler across a suite of test configurations that vary in job length, system size and workload intensity. The results demonstrate that the proposed algorithm consistently achieves competitive turnaround time in comparison to the baseline algorithms whilst not over-using resources or increasing server rental cost, often outperforming established heuristics in overall average performance. Each section of this report will describe the methodology, experimental setup, results, and key observations in detail while also highlighting where the custom scheduler performs well and identifying potential areas for further improvement. [1]

2 Problem Definition

In a distribution system and data centres, job scheduling is extremely critical to determining the responsiveness and efficiency of the entire system. The fundamental challenge is to assign incoming jobs with specific resource requirements to available servers with varying capabilities. Jobs must arrive dynamically and must be assigned to servers in real time without prior knowledge of future jobs. Each server differs in terms of available resources and processing power which makes it essential for scheduling algorithms to make intelligent decisions that align with performance and cost goals.

In this scenario, each job is defined by its required number of cores, memory, disk space and estimated runtime. Each server consists of fixed resource capacities and rental costs. The scheduler must identify the appropriate server that meets the job's resource requirements. However, assigning a job to an idle server may not always be the optimal approach if the server is under-utilised or leads to higher costs. Therefore, the scheduling strategy must balance resources matching with cost efficiency whilst also attaining performance goals.

The objective function for this scheduling problem focuses on three main goals:

- minimising average job turnaround time
- maximising server resource utilisation whilst not over-loading any server
- minimising the total rental cost

Turnaround time is the duration from the job's arrival to its completion and is a direct measure of user-perceived responsiveness. Low utilisation of the servers suggests inefficient use of available system resources. It is important to avoid overloading or underusing servers. The rental cost reflects the economic aspect of running the system with different server types incurring different usage charges. High cost is usually not favoured in any aspect but it is important for any expense to be justified with performance from all resources. Balancing these three objectives often requires trade-offs, especially when prioritising one metric may negatively impact the others.

3 Algorithm Description

The algorithm selects servers with the EJWT, reducing queue delays. Furthermore, it incorporates a resource over-provisioning penalty to discourage inefficient allocations where a job uses significantly less than the server's capacity. If no suitable server is available at the moment, the algorithm includes an efficient fallback mechanism that ensures jobs are still scheduled promptly without violating constraints. The algorithm operates in real-time and evaluates all servers that can satisfy a job's resource requirements and selects the server that minimises a weighted penalty function. [1]

When a new job first arrives, the algorithm first finds an available server that has zero wait time with enough cores, memory and disk space and schedules the job immediately. If there are no immediate servers available, the algorithm will estimate the time the job would have to wait before execution based on the current jobs queued or running on the server. Over-provisioning (assigning a job to a server that has more resources than the job actually needs which can lead to inefficient use of resources) is calculated by comparing the job's resources demands to the server's total capacity. This method is designed to discourage sending smaller jobs to larger servers therefore discouraging inefficient allocations. These two values are then combined in a weighted-cost formula. The lower values are preferred when selecting a server and dispatching the respective job.[1]

If no server is currently available to run the job immediately, the algorithm falls back on an **efficient queuing strategy**. Instead of waiting for idle servers or randomly retrying, the job is assigned to the server that would be able to process it the quickest. This ensures forward progress while minimising queue build-up and avoiding starvation. By evaluating the depth of queue and the estimated job duration, the fallback mechanism keeps turnaround times low even under heavy system load.[1]

To illustrate how the algorithm works consider a simple scenario with three servers:

- **Server A:** 4 cores, 16000 memory, 256000 disk
- **Server B:** 8 cores, 32000 memory, 64000 disk
- **Server C:** 2 cores, 4000 memory 16000 disk

Incoming jobs:

- **Job 1:** 2 cores, 4000 memory, 15000 disk, estimated runtime: 20
- **Job 2:** 1 core, 2000 memory, 2000 disk, estimated runtime: 15
- **Job 3:** 4 cores, 30000 memory, 100000 disk, estimated runtime: 30

Job 1's resource demands show that it can be scheduled to any of the three servers but the algorithm calculated that Server C is the fit since it has no current queue and minimal over-provisioning penalties in comparison to other available servers. The lightest Job 2 can also be assigned to Server C if it is immediately available. However, the fallback strategy would choose the next fastest available server such as Server A. Job 3 has higher resource demands and can only run on Server B due to its core requirement. The algorithm will continue to only allocate it there even if there is a queue since there are no other server that can take the job's demands.

This example shows how the algorithm balances immediate availability with resource matching. It avoids placing lightweight jobs on powerful servers (which could be used for heavier jobs later on) to preserve a balance between the three factors of turnaround time, resource utilisation and rental cost. The average results across all provided test configuration show that this method often matches in all key performance dimension and also outperforms in turnaround time.[1]

4 Implementation Details Including Data Structures

The scheduling algorithm was implemented in Python and integrated into the provided client framework to interact with the `ds-server` through a socket. Communication between the client and server is managed using a pair of helper functions: `send(sock, msg)` and `read_line(sock)`, which ensure that each message follows the correct protocol format and is parsed appropriately.[1, 2]

Jobs received from the server are implemented using Python dictionaries. Each job dictionary contains fields that indicate job ID, submit time, required cores, memory, disk, and estimated runtime. This structure allows for fast and readable access to individual attributes during the scheduling process. For example, resource requirements can be accessed directly as `job['cores']` or `job['memory']`, which simplifies the implementation of resource comparisons and score calculations.[1, 2]

Server information is stored as lists of strings parsed from the server's response lines. Each server entry contains several fields such as server type, server ID, state, and available resources. The list structure is sufficient for indexing specific values needed for comparison, such as `xlarge 4` for available cores. This lightweight representation supports the efficient request of information and the evaluation of multiple servers for optimal job scheduling.[1, 2]

To select the most suitable server for each job, the algorithm computes a composite score that combines the EJWT and a calculated over-provisioning penalty. These scores are stored using tuples in the form `(score, server)`. All such tuples are collected in a list called `scored_servers`, which is then sorted based on the score with the lowest value being favoured. This enables quick selection of the best server by simply taking the first item in the sorted list.[1]

The over-provisioning component of the score is calculated by subtracting the job resource requirements from the available server resources. The memory and disk values are divided by 1000 to normalise them with respect to the cores. This helps balance their contribution in the scoring formula so that the other factors do not overwhelm the bias with larger value weighting. This design prevents larger memory or disk values from dominating the selection decision ensuring that the chosen server is both timely and resource-efficient.[1]

The scheduling algorithm designed for this assignment balances simplicity with effectiveness by using basic data structures such as lists, dictionaries, and tuples. The use of these data structures allows for easy parsing and comparison of data to effectively sort and schedule jobs to their respective servers.[1]

5 Results and Evaluation

5.1 Simulation Setup

The scheduling algorithm is evaluated using the `ds-sim` discrete event simulator with a variety of XML-based workload configuration files. These configurations create varied job lengths (instant, short, medium, long and xlong), system loads (low, medium, high, alternating) and server pool sizes. The diverse configuration types allow testing on different aspects. These tests dictate compatibility of the scheduling algorithm to evaluate the balance between turnaround time, resource utilisation and rental cost of server usages.[1, 2]

The simulation is run using the custom scheduler `client.py` and compared against several baseline algorithms provided by the simulator.[1, 2] These including:

- **ATL** (All to Largest): dispatches all jobs to the largest server.
- **FF** (First Fit): assigns the first job to the first available server that can run it.
- **BF** (Best Fit): assigns to the server that maximises resource resources.
- **FC** (Fastest Completion): chooses the server that is capable of finishing the job the fastest.
- **FAFC** (Fastest Available Fastest Completion): hybrid of fastest first and fastest completion strategies.

Three metrics were used to compare the performance across the algorithms: **average turnaround time**, **average resource utilisation**, and **total rental cost**. The turnaround time was the main optimisation objective while maintaining a lower or similar utilisation and cost to the provided baseline algorithms.

5.2 Evaluation

The evaluation results show that the scheduling strategy designed for this assignment consistently outperforms most baseline algorithms across provided configurations. Table 1 summarises the average turnaround times, resource utilisation, and rental costs against the provided test configurations. The algorithm achieves the lowest overall average turnaround time of **1383.13**, while maintaining high resource utilisation (**74.54%**) and comparable rental cost (**526.05**). These results confirm that combining EJWT and resource over-provisioning provides an effective balance of performance and resource management.

Table 1: Average Metrics Across All Provided Configurations

Algorithm	Turnaround Time	Utilisation (%)	Rental Cost
ATL	259636.47	100.00	377.39
FF	1867.13	74.02	526.40
BF	2417.27	69.55	524.82
FC	306811.60	97.13	377.60
FAFC	1397.73	74.37	526.72
Mine	1383.13	74.54	526.05

Figure 1 shows a visualisation of job scheduling for `config-mermaid.xml` (a custom configuration file made with a limit of 500 jobs). Figure 1 shows that the jobs are distributed across available servers with minimal queuing and effective use of parallelism. Red bars indicate the waiting periods of the corresponding jobs and the staggered allocation across multiple servers highlights how the algorithm can dynamically schedule jobs in accordance to resource needs. The figure confirms how the strategy prevents bottlenecks and improves job performance under simulated workloads.

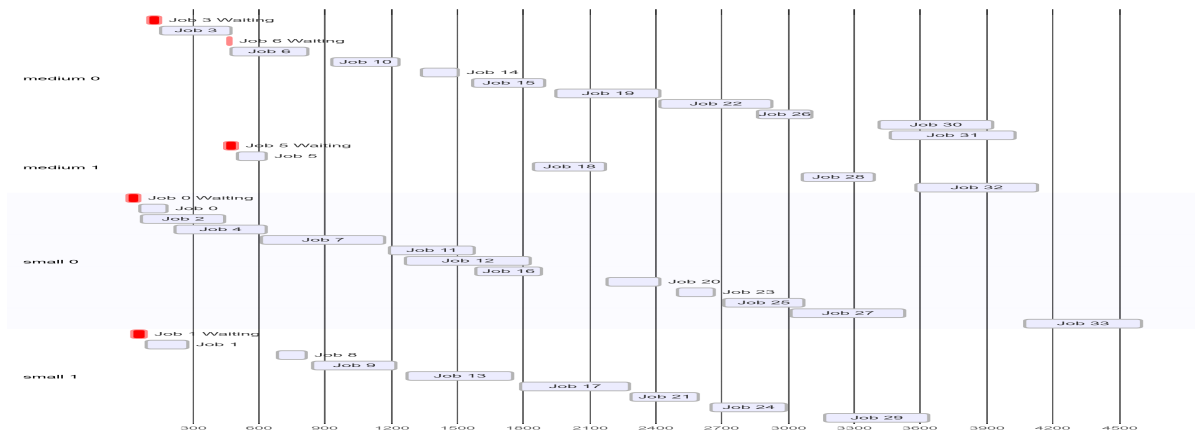


Figure 1: Job scheduling visualisation using `config-mermaid.xml`

The **ATL** strategy assigns every job to the largest available server. The proposed strategy dramatically reduces turnaround-time in comparison. Although ATL achieves maximum resource utilisation (obtaining an average utilisation of **100%**), it suffers from severe queuing and load imbalance due to the continuous use of the most powerful servers. This strategy results in an extremely high average turnaround time of **259636.47**. In contrast, The proposed strategy distributes jobs more intelligently where it reduces queue delays while still maintaining over **74%** average utilisation across multiple configurations.

When compared to **FF**, there are significant improvements in turnaround-time (**1383.13** vs. **1867.13**) while maintaining nearly identical resource usage (**74.54%** vs. **74.02%**) and rental cost (**526.40** vs. **526.05**). **FF** assigns the job to the first server that can accommodate it which often leads to suboptimal placements and local bottle-necking. Implementing the algorithm in `client.py` mitigates this by taking account queue times and resource waste before assigning jobs to the respective servers.

BF attempts to minimise leftover resources by placing each job on the most perfect-fitting server. Although this is efficient in maxing out resource usage, it often leads to fragmented resource availability and longer queue times. The current proposed scheduler tends to outperforms **BF** with a much lower average turnaround time (**1383.13** vs. **2417.27**) while still achieving higher utilisation (**74.54%** vs. **69.55%**). However, **BF** can achieve a lower cost by sacrificing responsiveness but not by much (**526.40** vs. **526.05**).

FC performs poorly in terms of turnaround time despite the higher utilisation. It chooses the server that would finish the job fastest if it started immediately. However, it proceeds without considering queue length or resource wastage. This leads to job congestion on the fastest servers and under-utilisation of less favourable servers. As a result, it yields a high turnaround time (**306811.60**), even worse than the already high **ATL**. The suggested approach avoids this by accounting for actual wait times and distributing the load more evenly.

Finally, the custom algorithm's closest competitor is **FAFC**. **FAFC** also uses a smart selection strategy and achieves similar results in terms of turnaround time (**1397.73** vs. **1383.13**) and rental cost. It is a hybrid scheduling strategy designed to balance responsiveness with performance. However, the custom algorithm consistently performs better across more configurations and achieves slightly higher resource utilisation (**74.54%** vs. **74.37%**). This suggests that incorporating the over-provisioning penalty in our cost function provides a marginal but consistent edge.

Advantages: The algorithm's combination of queue-time estimation and over-provisioning control provides a good balance in assigning jobs without significant wait times whilst also not wasting resources during runtime. Unlike **FF** or **BF**, which make simplistic placement decisions, the suggested method alters based on current system load and predicted server availability. Compared to **FC** and **FAFC**, the algorithm avoids aggressive overloading of fast servers, which leads to more balanced queue distribution.

It also exhibits stable behaviour across small- and large-scale environments with significant job limits and server pool size. The approach taken ensures better fairness and improved parallelism while **ATL** and **FC** either saturate a small number of servers or distribute jobs unevenly. Furthermore, the fallback mechanism guarantees that no job left unscheduled, even when immediate placement is not possible.

Limitations: The algorithm does not explicitly account for server processing speed when computing wait times that may affect performance in a more advanced system with varying compute power. Additionally, the over-provisioning penalty assumes static thresholds in some situation where either memory or disk wastage matters more.

Scalability: Efficient filtering and decision-making enables to the system to scale even if the number of servers were to increase. The use of dictionaries and heaps ensures that scheduling decisions remain fast even under thousands of job dispatches and larger server pool sizes. Furthermore, the strategy was designed for real-time use, where it has light computations that allow it to make decisions rapidly and repeatedly.

6 Summary

This assignment involved designing and implementing a new scheduling algorithm that would be comparable to the average capabilities of the baseline algorithms. The new algorithm optimises low turnaround time without sacrificing optimal resource utilisation and cost efficiency. By combining shortest estimated wait time with a resource over-provisioning penalty, the new scheduler made smarter server assignments in real-time. Overall, leading to improved performance across a variety of simulated workloads.

The evaluation showed that the custom scheduler outperformed baseline strategies such as **ATL**, **FF**, **BF**, **FC** and **FAFC**, particularly in terms of turnaround time. This adheres to the main goal of the assignment which was to optimise the turnaround time. It also matched or exceeded the performance of more sophisticated algorithms like **FAFC** while maintaining comparable resource usage and rental cost. These results demonstrate that queue-aware scheduling with over-provision control is an effective strategy for dynamic, multi-resource environments.

For future improvements, the algorithm could be extended by improving the over-provisioning penalty based on workload characteristics. For example, the weights assigned to excess memory, disk or cores could adjust depending on what is needed. Additionally, server configurations could be incorporated into both the EJWT calculation and the overall server scoring function. This would allow the scheduler to favour not just idle servers but also those capable of completing jobs faster due to higher computational resources.

References

- [1] Dickson Mai (emievar), "comp3100-6105-2025-s1-assignment-2-emievar." <https://github.com/MQU-COMP3100/comp3100-6105-2025-s1-assignment-2-emievar>. Accessed: 31/05/2025.
- [2] young-c-lee, Jayden King (JaydenKing32), "ds-sim." <https://github.com/distsys-MQ/ds-sim>. Accessed: 11/05/2025.